

## Static Nested Class :

It is class which we can declare at class level (inside a class)

If we declare a class inside an outer class with static modifier then it is called static nested class.

Example :

```
public class Outer
{
    static class Inner //static nested class
    {
    }
}
```

In order to access the static nested class, Outer class object is not required.

If a static nested class contains non static member then we need to create the object for static nested class but on the other hand if static nested class contains only static member then inner class object is not required.

A static nested class represents STATIC AREA so it can't access non static member of Outer class DIRECTLY.

```
//Programs :
package com.ravi.static_nested_class;

//static nested inner class
class BigOuter
{
    static class Nest //static nested class
    {
        public void go() //Instance Method of static inner class
        {
            System.out.println("Hello welcome to static nested class");
        }
    }
}

class Test11
{
    public static void main(String args[])
    {
        //Only static nested class object is created
        BigOuter.Nest inner = new BigOuter.Nest();
        inner.go();
    }
}

package com.ravi.static_nested_class;

class Outer
{
    static int x = 15;

    static class Inner
    {
        public void msg()
        {
            IO.println("x value is "+x);
        }
    }
}

class Test12
{
    public static void main(String args[])
    {
        Outer.Inner obj=new Outer.Inner();
        obj.msg();
    }
}

package com.ravi.static_nested_class;

class OuterTest
{
    static int x = 25;

    static class Inner
    {
        static void msg()
        {
            System.out.println("x value is "+x);
        }
    }
}

class Test13
{
    public static void main(String args[])
    {
        OuterTest.Inner.msg();
    }
}

package com.ravi.static_nested_class;

class OuterDemo
{
    int x=15; //error (not possible because try to access instance variable directly)

    static class Inner
    {
        void msg()
        {
            //IO.println("x value is "+x);
        }
    }
}

class Test14
{
    public static void main(String args[])
    {
        OuterDemo.Inner obj=new OuterDemo.Inner();
        obj.msg();
    }
}
```

## Declaring the classes at Method Level :

=====

At Method level we can declare two classes which are as follows :

- 1) Local OR Method Local inner class
- 2) Anonymous Inner class

Local OR Method Local Inner class :

-----

If we define a class with class name inside the method body then it is called Local OR Method Local Inner class.

Example :

```
public void method()
{
    class Local //Local OR Method Local Inner class
    {
    }
}
```

The scope of Method Local inner class is within the same method body that means it will be executed in the same method Stack Frame.

We can't access or use the method local inner class outside of the method body so basically it is used perform some operation within the method.

We can't apply any kind of access modifier on local inner class except abstract and final.

```
//Program
package com.ravi.method_local_class;

//program on method local inner class
class Outer
{
    private String x = "Outer class private data";

    public void accept()
    {
        String z = "local variable"; //final Or effectively final

        //z = "Modified value"; //not allowed

        class MyInner //Only final and abstract applicable
        {
            public void seeOuter()
            {
                System.out.println("Outer x is "+x);
                System.out.println("local variable z is : "+z);
            }
        }

        MyInner mi = new MyInner();
        mi.seeOuter();
    }
}

public class Test15
{
    public static void main(String args[])
    {
        Outer out = new Outer();
        out.accept();
    }
}

package com.ravi.method_local_class;

//local inner class we can't access outside of the method
class MyOuter
{
    private String z = "Outer class Data";

    public void doStuff()
    {
        String x = "local variable";

        class MyInner
        {
            String z = "CLASS instance variable";
            public void seeOuter()
            {
                System.out.println("Outer x is "+MyOuter.this.z);
                System.out.println("Class variable z is : "+this.z);
                System.out.println("Local variable is : "+x);
            }
        }

        MyInner mi = new MyInner(); //Object creation must be inside the method body
        mi.seeOuter();
    }
}

/* MyInner mi = new MyInner(); //Invalid because outside of the method body
*/

public class Test16
{
    public static void main(String args[])
    {
        MyOuter m = new MyOuter();
        m.doStuff();
    }
}
```

Note : Method Local inner class will be accessible within the same method body only.

## \*\*\*\* Anonymous inner class (Instant Implementation) :

If we declare a class inside a method without any name then it is called Anonymous inner class.

The main purpose of anonymous inner class to **extend a class OR to implement an interface** that means to create a **sub type**.

The anonymous inner class (class without any name) declaration and object creation by using new keyword both are done in the same line at the time of declaration of anonymous inner class.

For every anonymous inner class, .class file will be generated internally, java compiler will automatically generate \$1.class, \$2.class and so on

As a programmer we can write all types of static and non static members except constructor and we can't declare anonymous inner class as an abstract class

By using Anonymous inner class we can create only object at time so it is suitable for "Singleton" design pattern.

```
//Program :
package com.ravi.anonymous_inner_class;

class Vehicle
{
    public void run()
    {
        IO.println("Vehicle is running");
    }
}

public class AnonymousExample
{
    public static void main(String[] args)
    {
        //Anonymous inner class
        Vehicle car = new Vehicle()
        {
            @Override
            public void run()
            {
                IO.println("Car is running");
            }
        };

        car.run();
    }
}

package com.ravi.anonymous_inner_class;

abstract class Bird
{
    abstract public void fly();
}

public class AnonymousDemo {

    public static void main(String[] args)
    {
        //Anonymous inner class object
        Bird parrot = new Bird()
        {
            @Override
            public void fly()
            {
                IO.println("Parrot bird is flying");
            }
        };

        //Anonymous inner class object
        Bird sparrow = new Bird()
        {
            @Override
            public void fly()
            {
                IO.println("Sparrow bird is flying");
            }
        };

        parrot.fly(); sparrow.fly();
    }
}
```

## interface :



## interface JDK 1.7V

-----